

데이터 사이언스

2026-1학기

ICT융합학부 컴퓨터공학트랙

김대환

목차

1. 데이터 사이언스란?
2. 인과 관계 및 실험 (Causality and Experiments)
3. 데이터 사이언스 모듈
 1. 시퀀스
 2. 테이블
 3. 시각화
 4. 함수와 테이블

데이터 사이언스

데이터 사이언스 (1)

- 데이터 사이언스

- 단순히 데이터를 수집하는 것에 그치지 않고, 대규모의 다양한 데이터 세트(large and diverse data sets)로부터 유용한 결론(useful conclusions)을 도출해내는 학문
- 3단계 프로세스
 - **탐색(Exploration)**
 - 정보 내에 숨겨진 패턴(Patterns)을 식별하는 과정
 - 시각화 (Visualization), 기술 통계 (Descriptive Statistics)
 - **예측(Prediction)**
 - 이미 알고 있는 정보를 활용하여 우리가 알고자 하는 미지의 값에 대해 정보에 기반한 추측(Informed Guesses)을 하는 것
 - 머신러닝 (Machine learning), 최적화(Optimization)
 - **추론(Inference)**
 - 결론에 대한 확신의 정도를 정량화하는 과정 (얼마나 정확한가?)
 - 통계적 검정 (Statistical tests), 모델(Models)

데이터 사이언스 (2)

- 데이터 사이언스 분야

- 통계학 (Statistics)
 - 불완전한 정보(Incomplete Information)를 바탕으로 어떻게 강건한 결론(Robust Conclusions)을 내릴 것인가를 연구
- 컴퓨터 과학 (Computing/Programming)
 - 실제 응용 분야에서 발생하는 크고 다양한 데이터 세트에 분석 기술을 적용
 - 수치 데이터뿐만 아니라 텍스트, 이미지, 비디오, 센서 데이터 등 비정형 데이터 처리를 가능

- 도메인 지식의 중요성

- 데이터 과학은 각 구성 요소의 단순한 합 그 이상
- 특정 도메인(분야)에 대한 깊은 이해가 있어야만 적절한 질문을 던질 수 있어야 함
- 계산 및 추론 도구가 제공하는 답변을 해당 맥락에 맞게 올바르게 해석하는 능력이 필수

데이터 사이언스 (3)

■ 데이터(Data)란?

- 우리 주변 세계에 대한 묘사이며, 관찰을 통해 수집되어 컴퓨터에 저장된 정보를 의미
- 데이터 사이언스의 목표
 - 실제 수집하는 데이터(기후, 시장, 여론 등)는 대상을 불완전하게 묘사하는 경우가 많음
 - 부분적인 정보(Partial Information)를 가지고 신뢰할 수 있는 결론을 내리는 것

■ 데이터 분석의 핵심 도구

■ 계산 (Computation)

- 컴퓨터는 우리가 가진 모든 가용 정보를 활용해 결론을 도출하게 해 줌
- (예) 기후 변화 분석 시 단순히 평균 기온에만 집중하는 것이 아니라, 전체 기온 범위를 함께 고려하여 더 정교한 분석을 수행

■ 무작위성 (Randomness)

- 불완전한 정보가 채워질 수 있는 다양한 가능성을 고려함
- 데이터가 특정 방식으로 변할 것이라고 가정하는 대신, 무작위성을 활용해 관찰된 데이터와 일치하는 수많은 시나리오를 상상하고 분석

데이터 사이언스 (4)

- 통계학의 핵심

- 불완전한 정보(Incomplete Information)로부터 강건한 결론(Robust Conclusions)을 도출하는 방법을 연구
- 3개 핵심 문제
 - 가설 검정 (Testing Hypotheses)
 - 신뢰도 추정 (Estimating Confidence)
 - 미지수 예측(Predicting Unknown Quantities)

- 데이터 사이언스를 통한 통계 확장

- 빠른 컴퓨터와 인터넷을 통해 수백만 개의 기사, 데이터베이스, 멀티모달(음악, 사진, 비디오) 데이터 등 방대한 데이터 세트에 접근하고 분석
- 실제 데이터는 단순한 방정식이나 평균값(Average)만으로는 설명할 수 없는 흥미로운 변동성과 복잡성을 지님

데이터 사이언스 (5)

- 의사결정과 불확실성 감소
 - 데이터의 역할: 대규모 데이터 세트와 계산 도구를 활용함으로써 의사결정의 불확실성을 획기적으로 줄임
- 비판적 사고의 현대적 도구
 - 귀납적 추론(Inductive Reasoning): 비즈니스나 사회 과학의 결론은 명확히 입증되기보다 가용한 증거에 의해 지지될 뿐이며, 데이터 과학은 이를 위한 정밀하고 정량적인 근거를 제공
 - 신뢰할 수 있는 논증: 정보와 계산 능력에 대한 접근이 전례 없이 쉬워진 현대 사회에서, 데이터를 활용한 추론 기술이 없는 비판적 사고는 불완전

인과 관계 및 실험

인과 관계 및 실험 (1)

■ 관찰 연구의 기초 (Basics of Observational Study)

- 데이터를 관찰하기만 할 뿐, 데이터 생성 과정에 직접 개입하지 않고 결론을 내리는 연구 방식
- 핵심 구성 요소 3가지
 - 개체(Individuals): 연구 대상(사람, 국가, 주 등)
 - 처치(Treatment): 연구하고자 하는 관심 요인(예: 초콜릿 섭취, 사형제도 법안)
 - 결과(Outcome): 각 개체에서 측정된 수치(예: 심장병 지수, 살인율)

■ 연관성 (Association) vs 인과 관계 (Causality)

- 연관성: 처치와 결과 사이에 나타나는 모든 관계를 의미
- 인과 관계: 처치가 직접적으로 결과를 발생시키는 특정 관계를 의미

■ 인과 관계 확립 2단계

- 연관성 관찰: 먼저 데이터상에서 처치와 결과 사이의 관계를 발견
- 면밀한 분석: 추가적인 분석을 통해 해당 연관성이 단순한 우연인지, 실제 인과관계인지를 결정

인과 관계 및 실험 (2)

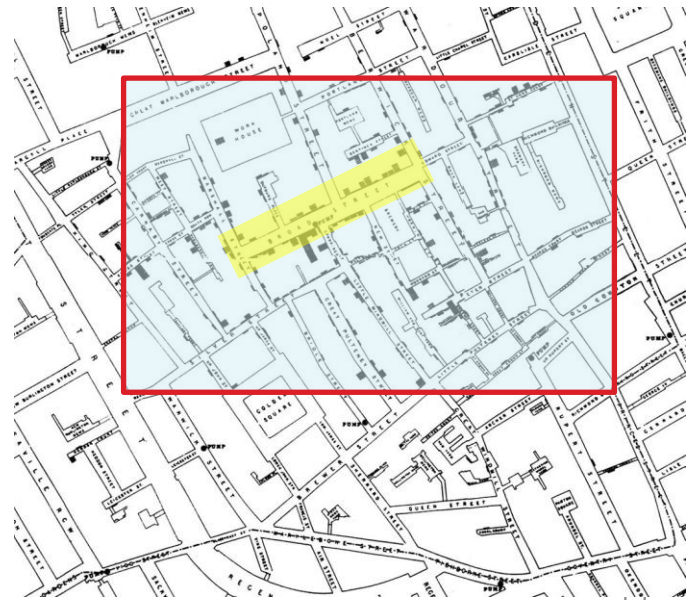
■ 관찰 (Observation) 및 시각화 (Visualization)

■ (예시) 존 스노우와 <브로드가 펌프 사건>

- 1850년대 런던은 세계에서 가장 부유, 위생 열악 → 원인 모를 콜레라 수만 명 사망
- 미아즈마 (Miasma) 이론 (가설) → 보이지 않는 독성 입자가 공기 통해 전파 믿음
- 존 스노우의 의심
 - 같은 공기를 마시는 이웃집 간에서 감염 여부에 대한 회의
 - 구토와 설사 증상을 근거로 **음용수 오염**을 원인으로 지목

■ 존 스노우의 지도 (데이터 시각화)

- 1854년 사망자 위치를 지도 위에 검은 바 (Bar)로 기록 → 급수 펌프의 위치를 표시
- 사망자들이 **브로드 가(Broad Street) 펌프** 주변에 집중적으로 밀집되어 있음을 시각적으로 확인



인과 관계 및 실험 (3)

- 관찰 (Observation) 및 시각화 (Visualization)

- (예시) 존 스노우와 <브로드가 펌프 사건> (Cont')

- 이상치 분석을 통한 가설 검증 (Analyzing anomalies)

- (펌프 근처의 생존자) 사자표 양조장(Lion Brewery) 직원들은 자체 우물물을 마시거나 맥주를 마셔 감염되지 않았음
 - (원거리의 사망자) 브로드 가 펌프의 물맛을 좋아해 멀리서도 물을 배달해 마신 사례(햄스테드의 수잔나 일리 부인)를 확인
 - (결과) 펌프 손잡이를 제거함으로써 전염 확산을 막았음 → 역학 조사에서 데이터 시각화의 힘을 보여준 상징적 사건

- 인과 관계를 향한 비교 방법론

- 비교 (Comparison): 처치와 결과 사이이 연관성을 찾기 위한 필수 과정

- 실험 설계 기초

- 처치군(Treatment group): 특정 요인(예: 오염된 물)에 노출된 그룹
 - 대조군(Control group): 노출되지 않은 그룹

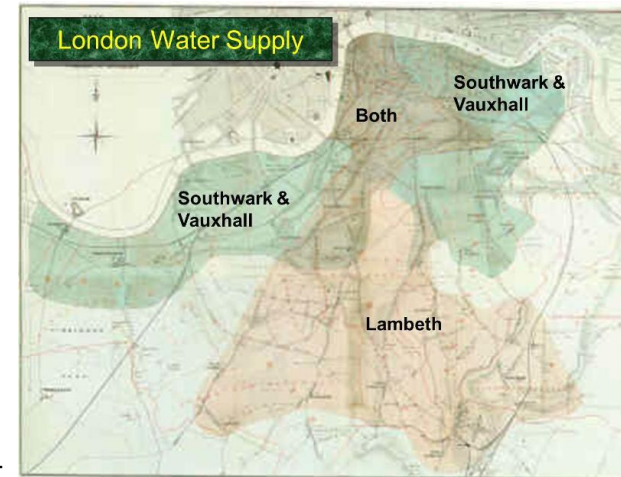
→ 두 그룹의 결과 차이는 연관성의 증거 → 인과 관계로 확정하기 위해서는 정밀한 분석 필요

인과 관계 및 실험 (4)

■ 관찰 (Observation) 및 시각화 (Visualization)

■ (예시) 존 스노우와 <브로드가 펌프 사건> (Cont')

- 실험 설계: 두 수도 회사의 비교
 - 런던의 특정 지역이 서로 다른 두 수도 회사로부터 물을 공급받는다는 점에 주목
 - 램베스(Lambeth) 회사: 상류에서 물을 끌어와 상대적으로 깨끗한 물을 공급
 - S&V 회사: 하류에서 물을 끌어와 오염된 물을 공급



■ 혼란 변수의 통제

- 두 그룹 사이에 수도 공급 외에는 차이가 없음
- (동질성) 두 회사 모두 부유한 층/가난한 층, 큰 집/작은 집, 다양한 직업군에 물을 공급
- (물리적 환경) 거주 조건이나 주변 물리적 환경에서도 차이 없음

→ 오직 수도 공급원만이 유일한 차이점

Supply Area	Number of houses	cholera deaths	deaths per 10,000 houses
S&V	40,046	1,263	315
Lambeth	26,107	98	37
Rest of London	256,423	1,422	59

→ S&V 공급 가구의 사망률은 램베스 공급 가구보다 약 10배 높음

인과 관계 및 실험 (5)

- **인과 관계 확립 및 교란 변수**

- **인과 관계 확립의 조건**

- (단일 차이의 원칙) 처치(Treatment) 외의 모든 조건이 유사한 두 그룹을 비교
 - (예) 처치군(Treatment Group): S&V사의 오염된 물을 공급받은 가구
 - 대조군(Control Group): 람베스사의 깨끗한 물을 공급받은 가구
 - 만약 두 그룹이 직업(예: 공장 노동자 여부) 등 다른 조건에서도 차이가 있었다면
 - ➔ 결과의 원인이 물 때문인지 직업 때문인지 판별하기 어려움

- **교란 변수 (Confounding Factors)**

- 처치 이외에 실험군과 대조군 사이에 존재하는 근본적인 차이점을 말하여, 결론 도출을 방해 요소
 - 관찰 연구에서 두 그룹이 처치 외의 다른 방식이 다르면 ➔ 인과관계에 대한 결론을 내리기 어려움

- **(사례분석) 커피와 폐암**

- 1960년대 연구 결과, 커피를 마시는 사람들의 폐암 발생률이 더 높게 나타남
 - (오류) 일부는 커피를 폐암의 원인으로 지목했으나, 이는 잘못된 인과관계
 - (교란 변수) 흡연(Smoking): 당시 커피를 즐기는 사람들은 담배를 피울 확률도 높았음
 - (결론) 커피와 폐암 사이에는 연관성이 있었으나, 실제 인과관계는 '흡연'과 '폐암' 사이에 존재

인과 관계 및 실험 (6)

- 무작위성 (Randomization)

- 무작위 대조 시험 (Randomized controlled trial: RCT)

- 대상자를 무작위로 처치군과 대조군에 배정하여, 처치 외의 모든 조건을 동일하게 유지함으로써 교란 변수를 차단
 - 실험 기법
 - 맹검 실험(Blind Experiment): 대상자가 자신이 어느 그룹에 속했는지 모르게 하여 심리적 요인을 배제
 - 플라세보(Placebo): 대조군에게 처치와 외형은 같으나 효과는 없는 가짜 약 등을 투여

- (사례 연구) 멕시코의 복지 보조금 (PROGRESA)

- 빈곤 가구 자녀의 학교 등록률과 건강 상태 개선을 위한 복지 프로그램의 효과 검증
 - 마을들을 무작위로 선정하여 일부(처치군)에만 정기적 등교 및 건강 관리 조건부로 현금을 지급
 - 결과: 무작위 배정 덕분에 교란 변수 없이 프로그램의 효과를 입증
 - 남학생: 등록률 73% → 77%로 상승.
 - 여학생: 등록률 67% → 약 75%로 대폭 상승

인과 관계 및 실험 (7)

- 무작위성 (Randomization) (2)

- 무작위 배정의 이점

- 자연 실험 vs. 진정한 무작위화: 존 스노우의 연구 같은 '자연 실험'도 강력하지만, 진정한 무작위화는 확률 법칙을 정밀하게 따라야 함
 - 수학적 통제: 무작위화는 두 그룹이 우연히 다를 가능성을 수학적으로 계산할 수 있게 해주며, 처치 효과에 대해 정밀한 수학적 진술을 가능케 함

- 무작위 배정이 불가능한 경우

- 윤리적/현실적 제약: 임산부의 음주 영향 연구처럼 무작위 배정이 비윤리적이거나 불가능한 상황이 존재
 - (대응책) 이러한 경우 **관찰 연구**를 수행할 수밖에 없으며, 이때는 **연관성**은 찾을 수 있어도 **인과관계**를 입증하기는 훨씬 어려움 → 반드시 교란 변수에 대해 극도로 주의

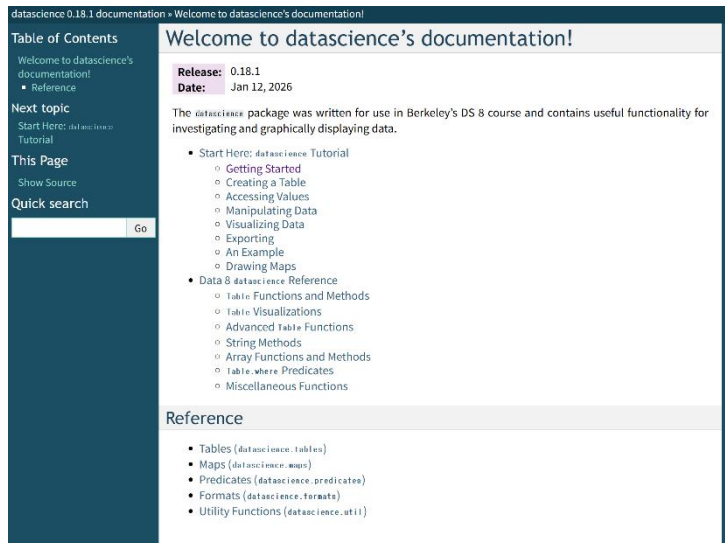
➔ 인과관계를 밝히고 싶다면 가능하다면 RCT를 수행하고, 불가능하여 관찰 연구를 할 때는 교란 변수라는 함정을 항상 경계

데이터 사이언스 모듈

데이터 사이언스 모듈 (1)

■ 데이터 사이언스 모듈

- Berkeley에서 개발한 자체 라이브러리로, 테이블 형태의 데이터를 조작하고 시각화하는 데 사용
- Pandas보다 직관적인 문법을 제공하여 초보자가 표(Table) 데이터를 다루고(group, join, sort 등) 그래프를 그리는 데 최적화
- 계산적 및 추론적 사고 강의 자료 사이트 - <https://inferentialthinking.com/>
- 데이터 사이언스 모듈 documentation 사이트 - <https://data8.org/datascience/>
- 데이터 사이언스 소스 Github 사이트 - <https://github.com/data-8/materials-sp26/tree/main/lec>
- datascience 설치 (주피터 노트북 - **!pip install datascience**) → **from datascience import ***



Getting Started

The most important functionality in the package is the `Table` class, which is the structure used to represent columns of data. First, load the class:

```
In [1]: from datascience import Table
```

In the IPython notebook, type `Table.` followed by the TAB-key to see a list of members.

Note that for the Data Science 8 class we also import additional packages and settings for all assignments and labs. This is so that plots and other available packages mirror the ones in the textbook more closely. The exact code we use is:

```
# HIDDEN
import matplotlib
matplotlib.use('Agg')
from datascience import Table
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np
plt.style.use('fivethirtyeight')
```

데이터 사이언스 모듈 (2) – 시퀀스

■ 시퀀스 (1)

■ 컬렉션 (Collection)

- 여러 값들을 하나로 묶어 단일한 이름으로 참조할 수 있게 만든 조직화된 구조
- 한 번의 계산으로 많은 양의 데이터를 동시에 처리

■ 배열 생성과 활용

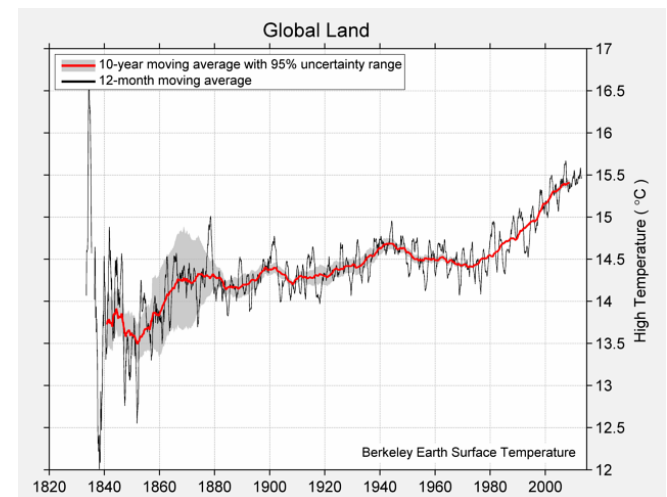
- (배열 생성) `make_array` 함수를 호출하여 여러 값을 순차적인 컬렉션인 배열
- `sum(array)`: 컬렉션 내의 모든 값의 합계를 계산
- `len(array)`: 컬렉션에 포함된 값의 개수(길이)를 계산

```
baseline_high = 14.48
highs = make_array(baseline_high - 0.880, baseline_high - 0.093,
                   baseline_high + 0.105, baseline_high + 0.684)
highs
```

```
array([13.6 , 14.387, 14.585, 15.164])
```

```
sum(highs)/len(highs)
```

```
14.434000000000001
```



데이터 사이언스 모듈 (3) – 시퀀스

■ 시퀀스 (2)

■ 배열 특성

- 단일 데이터 타입: 배열은 숫자뿐만 아니라 문자열(Strings) 등도 포함할 수 있으나, 하나의 배열에는 반드시 한 가지 종류의 데이터만 담음
- 브로드캐스팅(Broadcasting): 배열에 단일 숫자를 결합하여 연산하면, 배열 내의 모든 요소에 각각 해당 연산이 적용

```
english_parts_of_speech = make_array("noun", "pronoun", "verb", "adverb", "adjective", "english_parts_of_speech
```

```
array(['noun', 'pronoun', 'verb', 'adverb', 'adjective', 'conjunction',  
      'preposition', 'interjection'], dtype='<U12')
```

```
baseline_high = 14.48  
highs = make_array(baseline_high - 0.880,  
                   baseline_high - 0.093,  
                   baseline_high + 0.105,  
                   baseline_high + 0.684)  
  
highs
```

```
array([13.6 , 14.387, 14.585, 15.164])
```

```
(9/5) * highs + 32
```

```
array([56.48 , 57.8966, 58.253 , 59.2952])
```

$$(9/5) * \begin{matrix} 13.6 \\ 14.387 \\ 14.585 \\ 15.164 \end{matrix} + 32 = \begin{matrix} (9/5) * 13.6 + 32 \\ (9/5) * 14.387 + 32 \\ (9/5) * 14.585 + 32 \\ (9/5) * 15.164 + 32 \end{matrix} = \begin{matrix} 56.48 \\ 57.8966 \\ 58.253 \\ 59.2952 \end{matrix}$$

highs.size 배열 내 총 개수

4

highs.sum() 모든 요소의 합

57.736000000000004

highs.mean() 모든 요소의 평균

14.434000000000001

데이터 사이언스 모듈 (4) – 시퀀스

■ 넘파이 함수들

Function	Description
<code>np.prod</code>	Multiply all elements together
<code>np.sum</code>	Add all elements together
<code>np.all</code>	Test whether all elements are true values (non-zero numbers are true)
<code>np.any</code>	Test whether any elements are true values (non-zero numbers are true)
<code>np.count_nonzero</code>	Count the number of non-zero elements

Function	Description
<code>np.diff</code>	Difference between adjacent elements
<code>np.round</code>	Round each number to the nearest integer (whole number)
<code>np.cumprod</code>	A cumulative product: for each element, multiply all elements so far
<code>np.cumsum</code>	A cumulative sum: for each element, add all elements so far
<code>np.exp</code>	Exponentiate each element
<code>np.log</code>	Take the natural logarithm of each element
<code>np.sqrt</code>	Take the square root of each element
<code>np.sort</code>	Sort the elements

Function	Description
<code>np.char.lower</code>	Lowercase each element
<code>np.char.upper</code>	Uppercase each element
<code>np.char.strip</code>	Remove spaces at the beginning or end of each element
<code>np.char.isalpha</code>	Whether each element is only letters (no numbers or symbols)
<code>np.char.isnumeric</code>	Whether each element is only numeric (no letters)

Function	Description
<code>np.char.count</code>	Count the number of times a search string appears among the elements of an array
<code>np.char.find</code>	The position within each element that a search string is found first
<code>np.char.rfind</code>	The position within each element that a search string is found last
<code>np.char.startswith</code>	Whether each element starts with the search string

데이터 사이언스 모듈 (5) - 테이블

■ 테이블 (Tables)

- 열 (Columns): 데이터 세트의 특정 속성(Attribute)을 설명하며, 각 열은 고유한 이름을 가집니다. 동일한 열의 데이터는 모두 같은 종류
- 행 (Rows): 데이터 세트의 개별 개체(Individual)에 대한 모든 정보
- 핵심 메소드들
 - '테이블이름.메소드(인자)' 형식으로 호출
 - 원본 테이블은 변경하지 않고, 새로운 테이블 생성

메서드	기능	예시
<code>show(n)</code>	처음 n개의 행을 화면에 출력	<code>cones.show(2)</code>
<code>select('L1', 'L2')</code>	지정한 열(Label)들만 포함하는 새 테이블을 생성	<code>cones.select('Flavor', 'Price')</code>
<code>drop('L1')</code>	특정 열을 제외한 새 테이블을 생성	<code>cones.drop('Color')</code>
<code>sort('L')</code>	지정한 열을 기준으로 행을 오름차순 정렬	<code>cones.sort('Price')</code>
<code>where('L', value)</code>	특정 조건을 만족하는 행들만 선택	<code>cones.where('Flavor', 'chocolate')</code>

데이터 사이언스 모듈 (6) - 테이블

■ 테이블 (Tables) (2) – 메소드 사용

- <https://github.com/data-8/materials-sp26/blob/main/lec/lec03/cones.csv>

cones	cones.show(2)	cones.select('Flavor')	cones.select('Flavor', 'Price')	cones.drop('Color')																																																																				
<table><tr><th>Flavor</th><th>Color</th><th>Price</th></tr><tr><td>strawberry</td><td>pink</td><td>3.55</td></tr><tr><td>chocolate</td><td>light brown</td><td>4.75</td></tr><tr><td>chocolate</td><td>dark brown</td><td>5.25</td></tr><tr><td>strawberry</td><td>pink</td><td>5.25</td></tr><tr><td>chocolate</td><td>dark brown</td><td>5.25</td></tr><tr><td>bubblegum</td><td>pink</td><td>4.75</td></tr></table>	Flavor	Color	Price	strawberry	pink	3.55	chocolate	light brown	4.75	chocolate	dark brown	5.25	strawberry	pink	5.25	chocolate	dark brown	5.25	bubblegum	pink	4.75	<table><tr><th>Flavor</th><th>Color</th><th>Price</th></tr><tr><td>strawberry</td><td>pink</td><td>3.55</td></tr><tr><td>chocolate</td><td>light brown</td><td>4.75</td></tr><tr><td colspan="3">... (4 rows omitted)</td></tr></table>	Flavor	Color	Price	strawberry	pink	3.55	chocolate	light brown	4.75	... (4 rows omitted)			<table><tr><th>Flavor</th></tr><tr><td>strawberry</td></tr><tr><td>chocolate</td></tr><tr><td>chocolate</td></tr><tr><td>strawberry</td></tr><tr><td>chocolate</td></tr><tr><td>bubblegum</td></tr></table>	Flavor	strawberry	chocolate	chocolate	strawberry	chocolate	bubblegum	<table><tr><th>Flavor</th><th>Price</th></tr><tr><td>strawberry</td><td>3.55</td></tr><tr><td>chocolate</td><td>4.75</td></tr><tr><td>chocolate</td><td>5.25</td></tr><tr><td>strawberry</td><td>5.25</td></tr><tr><td>chocolate</td><td>5.25</td></tr><tr><td>bubblegum</td><td>4.75</td></tr></table>	Flavor	Price	strawberry	3.55	chocolate	4.75	chocolate	5.25	strawberry	5.25	chocolate	5.25	bubblegum	4.75	<table><tr><th>Flavor</th><th>Price</th></tr><tr><td>strawberry</td><td>3.55</td></tr><tr><td>chocolate</td><td>4.75</td></tr><tr><td>chocolate</td><td>5.25</td></tr><tr><td>strawberry</td><td>5.25</td></tr><tr><td>chocolate</td><td>5.25</td></tr><tr><td>bubblegum</td><td>4.75</td></tr></table>	Flavor	Price	strawberry	3.55	chocolate	4.75	chocolate	5.25	strawberry	5.25	chocolate	5.25	bubblegum	4.75
Flavor	Color	Price																																																																						
strawberry	pink	3.55																																																																						
chocolate	light brown	4.75																																																																						
chocolate	dark brown	5.25																																																																						
strawberry	pink	5.25																																																																						
chocolate	dark brown	5.25																																																																						
bubblegum	pink	4.75																																																																						
Flavor	Color	Price																																																																						
strawberry	pink	3.55																																																																						
chocolate	light brown	4.75																																																																						
... (4 rows omitted)																																																																								
Flavor																																																																								
strawberry																																																																								
chocolate																																																																								
chocolate																																																																								
strawberry																																																																								
chocolate																																																																								
bubblegum																																																																								
Flavor	Price																																																																							
strawberry	3.55																																																																							
chocolate	4.75																																																																							
chocolate	5.25																																																																							
strawberry	5.25																																																																							
chocolate	5.25																																																																							
bubblegum	4.75																																																																							
Flavor	Price																																																																							
strawberry	3.55																																																																							
chocolate	4.75																																																																							
chocolate	5.25																																																																							
strawberry	5.25																																																																							
chocolate	5.25																																																																							
bubblegum	4.75																																																																							
<pre>no_colors = cones.drop('Color') no_colors</pre>	cones.sort('Price')	cones.sort('Price', descending=True)	cones.where('Flavor', 'chocolate')																																																																					
<table><tr><th>Flavor</th><th>Price</th></tr><tr><td>strawberry</td><td>3.55</td></tr><tr><td>chocolate</td><td>4.75</td></tr><tr><td>chocolate</td><td>5.25</td></tr><tr><td>strawberry</td><td>5.25</td></tr><tr><td>chocolate</td><td>5.25</td></tr><tr><td>bubblegum</td><td>4.75</td></tr></table>	Flavor	Price	strawberry	3.55	chocolate	4.75	chocolate	5.25	strawberry	5.25	chocolate	5.25	bubblegum	4.75	<table><tr><th>Flavor</th><th>Color</th><th>Price</th></tr><tr><td>strawberry</td><td>pink</td><td>3.55</td></tr><tr><td>chocolate</td><td>light brown</td><td>4.75</td></tr><tr><td>bubblegum</td><td>pink</td><td>4.75</td></tr><tr><td>chocolate</td><td>dark brown</td><td>5.25</td></tr><tr><td>strawberry</td><td>pink</td><td>5.25</td></tr><tr><td>chocolate</td><td>dark brown</td><td>5.25</td></tr></table>	Flavor	Color	Price	strawberry	pink	3.55	chocolate	light brown	4.75	bubblegum	pink	4.75	chocolate	dark brown	5.25	strawberry	pink	5.25	chocolate	dark brown	5.25	<table><tr><th>Flavor</th><th>Color</th><th>Price</th></tr><tr><td>chocolate</td><td>dark brown</td><td>5.25</td></tr><tr><td>strawberry</td><td>pink</td><td>4.75</td></tr><tr><td>chocolate</td><td>light brown</td><td>4.75</td></tr><tr><td>strawberry</td><td>pink</td><td>3.55</td></tr></table>	Flavor	Color	Price	chocolate	dark brown	5.25	strawberry	pink	4.75	chocolate	light brown	4.75	strawberry	pink	3.55	<table><tr><th>Flavor</th><th>Color</th><th>Price</th></tr><tr><td>chocolate</td><td>light brown</td><td>4.75</td></tr><tr><td>chocolate</td><td>dark brown</td><td>5.25</td></tr><tr><td>chocolate</td><td>dark brown</td><td>5.25</td></tr></table>	Flavor	Color	Price	chocolate	light brown	4.75	chocolate	dark brown	5.25	chocolate	dark brown	5.25	<pre>cones.where('Flavor', 'Chocolate')</pre> <table><tr><th>Flavor</th><th>Color</th><th>Price</th></tr></table>	Flavor	Color	Price			
Flavor	Price																																																																							
strawberry	3.55																																																																							
chocolate	4.75																																																																							
chocolate	5.25																																																																							
strawberry	5.25																																																																							
chocolate	5.25																																																																							
bubblegum	4.75																																																																							
Flavor	Color	Price																																																																						
strawberry	pink	3.55																																																																						
chocolate	light brown	4.75																																																																						
bubblegum	pink	4.75																																																																						
chocolate	dark brown	5.25																																																																						
strawberry	pink	5.25																																																																						
chocolate	dark brown	5.25																																																																						
Flavor	Color	Price																																																																						
chocolate	dark brown	5.25																																																																						
strawberry	pink	4.75																																																																						
chocolate	light brown	4.75																																																																						
strawberry	pink	3.55																																																																						
Flavor	Color	Price																																																																						
chocolate	light brown	4.75																																																																						
chocolate	dark brown	5.25																																																																						
chocolate	dark brown	5.25																																																																						
Flavor	Color	Price																																																																						

데이터 사이언스 모듈 (7) - 테이블

■ (실전 사례) NBA 선수 연봉 분석

- https://github.com/data-8/materials-sp26/blob/main/lec/lec03/nba_salaries.csv
- `nba_salaries = Table.read_table('nba_salaries.csv')`

1	rank	name	position	team	salary	season
2	1	Shaquille O'Neal	C	Los Angeles Lakers	17142000	2000
3	2	Kevin Garnett	PF	Minnesota Timberwolves	16806000	2000
4	3	Alonzo Mourning	C	Miami Heat	15004000	2000
5	4	Juwan Howard	PF	Washington Wizards	15000000	2000
6	5	Scottie Pippen	SF	Portland Trail Blazers	14795000	2000
7	6	Karl Malone	PF	Utah Jazz	14000000	2000
8	7	Larry Johnson	F	New York Knicks	11910000	2000
9	8	Gary Payton	PG	Seattle SuperSonics	11020000	2000
10	9	Rasheed Wallace	PF	Portland Trail Blazers	10800000	2000
11	10	Shawn Kemp	C	Cleveland Cavaliers	10780000	2000
12	11	Damon Stoudamire	PG	Portland Trail Blazers	10125000	2000
13	12	Antonio McDyess	PF	Denver Nuggets	9900000	2000
14	13	Antoine Walker	PF	Boston Celtics	9000000	2000
15	14	Shareef Abdur-Rahim	PF	Vancouver Grizzlies	9000000	2000

Column Label	Description
PLAYER	Player's name
POSITION	Player's position on team
TEAM	Team name
SALARY	Player's salary in 2015-2016, in millions of dollars

데이터 사이언스 모듈 (8) - 테이블

- (실전 사례) NBA 선수 연봉 분석 (2)
 - 특정 데이터 찾기: `nba.where('PLAYER', 'Stephen Curry')`
 - 그룹 추출: `nba.where('TEAM', 'Golden State Warriors')`
 - 순위 확인: `nba.sort('SALARY', descending=True)`

```
nba.where('PLAYER', 'Stephen Curry')
```

PLAYER	POSITION	TEAM	SALARY
Stephen Curry	PG	Golden State Warriors	11.3708

```
warriors = nba.where('TEAM', 'Golden State Warriors')  
warriors
```

PLAYER	POSITION	TEAM	SALARY
Klay Thompson	SG	Golden State Warriors	15.501
Draymond Green	PF	Golden State Warriors	14.2609
Andrew Bogut	C	Golden State Warriors	13.8
Andre Iguodala	SF	Golden State Warriors	11.7105
Stephen Curry	PG	Golden State Warriors	11.3708
Jason Thompson	PF	Golden State Warriors	7.00847
Shaun Livingston	PG	Golden State Warriors	5.54373
Harrison Barnes	SF	Golden State Warriors	3.8734
Marreese Speights	C	Golden State Warriors	3.815
Leandro Barbosa	SG	Golden State Warriors	2.5

... (4 rows omitted)

```
warriors.show()
```

PLAYER	POSITION	TEAM	SALARY
Klay Thompson	SG	Golden State Warriors	15.501
Draymond Green	PF	Golden State Warriors	14.2609
Andrew Bogut	C	Golden State Warriors	13.8
Andre Iguodala	SF	Golden State Warriors	11.7105
Stephen Curry	PG	Golden State Warriors	11.3708

```
nba.sort('SALARY')
```

PLAYER	POSITION	TEAM	SALARY
Thanasis Antetokounmpo	SF	New York Knicks	0.030888
Jordan McRae	SG	Phoenix Suns	0.049709
Cory Jefferson	PF	Phoenix Suns	0.049709
Elliot Williams	SG	Memphis Grizzlies	0.055722
Orlando Johnson	SG	Phoenix Suns	0.055722
Phil Pressey	PG	Phoenix Suns	0.055722
Keith Appling	PG	Orlando Magic	0.061776

```
nba.sort('SALARY', descending=True)
```

PLAYER	POSITION	TEAM	SALARY
Kobe Bryant	SF	Los Angeles Lakers	25
Joe Johnson	SF	Brooklyn Nets	24.8949
LeBron James	SF	Cleveland Cavaliers	22.9705
Carmelo Anthony	SF	New York Knicks	22.875
Dwight Howard	C	Houston Rockets	22.3594
Chris Bosh	PF	Miami Heat	22.1927
Chris Paul	PG	Los Angeles Clippers	21.4687
Kevin Durant	SF	Oklahoma City Thunder	20.1586

데이터 사이언스 모듈 (9) - 테이블

■ 행 정렬 (1)

- https://github.com/data-8/materials-sp26/blob/main/lec/lec05/nba_salaries.csv

Column Label	Description
PLAYER	Player's name
POSITION	Player's position on team
TEAM	Team name
'15-'16 SALARY	Player's salary in 2015-2016, in millions of dollars

```
# This table can be found online: https://www.statcrunch.com/app/index.php?dataid=184334
nba_salaries = Table.read_table(path_data + 'nba_salaries.csv')
nba_salaries
```

PLAYER	POSITION	TEAM	'15-'16 SALARY
Paul Millsap	PF	Atlanta Hawks	18.6717
Al Horford	C	Atlanta Hawks	12
Tiago Splitter	C	Atlanta Hawks	9.75625
Jeff Teague	PG	Atlanta Hawks	8
Kyle Korver	SG	Atlanta Hawks	5.74648
Thabo Sefolosha	SF	Atlanta Hawks	4
Mike Scott	PF	Atlanta Hawks	3.33333
Kent Bazemore	SF	Atlanta Hawks	2
Dennis Schroder	PG	Atlanta Hawks	1.7634
Tim Hardaway Jr.	SG	Atlanta Hawks	1.30452

... (407 rows omitted)

```
nba_salaries.show(3)
```

PLAYER	POSITION	TEAM	'15-'16 SALARY
Paul Millsap	PF	Atlanta Hawks	18.6717
Al Horford	C	Atlanta Hawks	12
Tiago Splitter	C	Atlanta Hawks	9.75625

... (414 rows omitted)

```
nba_salaries.sort('PLAYER').show(5)
```

PLAYER	POSITION	TEAM	'15-'16 SALARY
Aaron Brooks	PG	Chicago Bulls	2.25
Aaron Gordon	PF	Orlando Magic	4.17168
Aaron Harrison	SG	Charlotte Hornets	0.525093
Adreian Payne	PF	Minnesota Timberwolves	1.93884
Al Horford	C	Atlanta Hawks	12

... (412 rows omitted)

데이터 사이언스 모듈 (9) - 테이블

■ 행 정렬 (2)

```
nba = nba_salaries.relabeled("'15-'16 SALARY", 'SALARY')
nba.sort('SALARY')
```

PLAYER	POSITION	TEAM	SALARY
Thanasis Antetokounmpo	SF	New York Knicks	0.030888
Jordan McRae	SG	Phoenix Suns	0.049709
Cory Jefferson	PF	Phoenix Suns	0.049709
Elliot Williams	SG	Memphis Grizzlies	0.055722
Orlando Johnson	SG	Phoenix Suns	0.055722
Phil Pressey	PG	Phoenix Suns	0.055722
Keith Appling	PG	Orlando Magic	0.061776
Sean Kilpatrick	SG	Denver Nuggets	0.099418
Erick Green	PG	Utah Jazz	0.099418
Jeff Ayres	PF	Los Angeles Clippers	0.111444

... (407 rows omitted)

열 이름 변경

```
nba.sort('SALARY', descending=True)
```

PLAYER	POSITION	TEAM	SALARY
Kobe Bryant	SF	Los Angeles Lakers	25
Joe Johnson	SF	Brooklyn Nets	24.8949
LeBron James	SF	Cleveland Cavaliers	22.9705
Carmelo Anthony	SF	New York Knicks	22.875
Dwight Howard	C	Houston Rockets	22.3594
Chris Bosh	PF	Miami Heat	22.1927
Chris Paul	PG	Los Angeles Clippers	21.4687
Kevin Durant	SF	Oklahoma City Thunder	20.1586
Derrick Rose	PG	Chicago Bulls	20.0931
Dwyane Wade	SG	Miami Heat	20

... (407 rows omitted)

정렬 ('SALARY' 기준)

데이터 사이언스 모듈 (10) - 테이블

■ 행 가져오기

```
nba
```

PLAYER	POSITION	TEAM	SALARY
Paul Millsap	PF	Atlanta Hawks	18.6717
Al Horford	C	Atlanta Hawks	12
Tiago Splitter	C	Atlanta Hawks	9.75625
Jeff Teague	PG	Atlanta Hawks	8
Kyle Korver	SG	Atlanta Hawks	5.74648
Thabo Sefolosha	SF	Atlanta Hawks	4
Mike Scott	PF	Atlanta Hawks	3.33333
Kent Bazemore	SF	Atlanta Hawks	2
Dennis Schroder	PG	Atlanta Hawks	1.7634
Tim Hardaway Jr.	SG	Atlanta Hawks	1.30452

... (407 rows omitted)

```
nba.take(0)
```

PLAYER	POSITION	TEAM	SALARY
Paul Millsap	PF	Atlanta Hawks	18.6717

첫번째 행 선택

```
nba.take(np.arange(3, 6))
```

PLAYER	POSITION	TEAM	SALARY
Jeff Teague	PG	Atlanta Hawks	8
Kyle Korver	SG	Atlanta Hawks	5.74648
Thabo Sefolosha	SF	Atlanta Hawks	4

3~6번 행 선택

```
nba.sort('SALARY', descending=True).take(np.arange(5))
```

PLAYER	POSITION	TEAM	SALARY
Kobe Bryant	SF	Los Angeles Lakers	25
Joe Johnson	SF	Brooklyn Nets	24.8949
LeBron James	SF	Cleveland Cavaliers	22.9705
Carmelo Anthony	SF	New York Knicks	22.875
Dwight Howard	C	Houston Rockets	22.3594

연봉 높은 순서로 5개의 행 선택

데이터 사이언스 모듈 (11) - 테이블

■ where 메소드

- 특정 조건을 만족하는 행만 포함하는 새로운 테이블을 반환
- `original_table.where(column_label, are.condition)`
 - 'are' : 어떤 기준으로 선택 결정
 - (예) `nba.where('SALARY', are.above(10))` → 연봉 열에서 10보다 큰(above) 행만 선택

```
nba.where('SALARY', are.above(10))
```

PLAYER	POSITION	TEAM	SALARY
Paul Millsap	PF	Atlanta Hawks	18.6717
Al Horford	C	Atlanta Hawks	12
Joe Johnson	SF	Brooklyn Nets	24.8949
Thaddeus Young	PF	Brooklyn Nets	11.236
Al Jefferson	C	Charlotte Hornets	13.5
Nicolas Batum	SG	Charlotte Hornets	13.1253
Kemba Walker	PG	Charlotte Hornets	12
Derrick Rose	PG	Chicago Bulls	20.0931
Jimmy Butler	SG	Chicago Bulls	16.4075
Joakim Noah	C	Chicago Bulls	13.4

... (59 rows omitted)

연봉 10이 넘는 행만 선택

```
nba.where('SALARY', are.above(10)).sort('SALARY')
```

PLAYER	POSITION	TEAM	SALARY
DeMar DeRozan	SG	Toronto Raptors	10.05
Gerald Wallace	SF	Philadelphia 76ers	10.1059
Luol Deng	SF	Miami Heat	10.1516
Monta Ellis	SG	Indiana Pacers	10.3
Wilson Chandler	SF	Denver Nuggets	10.4494
Brendan Haywood	C	Cleveland Cavaliers	10.5225
Jrue Holiday	PG	New Orleans Pelicans	10.5955
Tyreke Evans	SG	New Orleans Pelicans	10.7346
Marcin Gortat	C	Washington Wizards	11.2174
Thaddeus Young	PF	Brooklyn Nets	11.236

... (59 rows omitted)

연봉 10이 넘는 행만 선택 후 정렬

데이터 사이언스 모듈 (12) - 테이블

where 메소드 (2) - 특정 행 선택

```
nba.where('PLAYER', are.equal_to('Stephen Curry'))
```

PLAYER	POSITION	TEAM	SALARY
Stephen Curry	PG	Golden State Warriors	11.3708

PLAYER 중 'Stephen Curry' 동일 행 선택

```
nba.where('TEAM', are.equal_to('Golden State Warriors')).show()
```

PLAYER	POSITION	TEAM	SALARY
Klay Thompson	SG	Golden State Warriors	15.501
Draymond Green	PF	Golden State Warriors	14.2609
Andrew Bogut	C	Golden State Warriors	13.8
Andre Iguodala	SF	Golden State Warriors	11.7105
Stephen Curry	PG	Golden State Warriors	11.3708
Jason Thompson	PF	Golden State Warriors	7.00847
Shaun Livingston	PG	Golden State Warriors	5.54373
Harrison Barnes	SF	Golden State Warriors	2.8724

TEAM 중 'Golden State Warriors' 동일 행 모두 보여주기

```
nba.where('TEAM', 'Denver Nuggets') # equivalent to nb
```

PLAYER	POSITION	TEAM	SALARY
Danilo Gallinari	SF	Denver Nuggets	14
Kenneth Faried	PF	Denver Nuggets	11.236
Wilson Chandler	SF	Denver Nuggets	10.4494
JJ Hickson	C	Denver Nuggets	5.6135
Jameer Nelson	PG	Denver Nuggets	4.345
Will Barton	SF	Denver Nuggets	3.53333
Emmanuel Mudiay	PG	Denver Nuggets	3.10224

TEAM 중 'Denver Nuggets' 선택

데이터 사이언스 모듈 (13) - 테이블

■ where 메소드 (3) – 다중 특징

```
nba.where('POSITION', 'PG').where('SALARY', are.above(15))
```

PLAYER	POSITION	TEAM	SALARY
Derrick Rose	PG	Chicago Bulls	20.0931
Kyrie Irving	PG	Cleveland Cavaliers	16.4075
Chris Paul	PG	Los Angeles Clippers	21.4687
Russell Westbrook	PG	Oklahoma City Thunder	16.7442
John Wall	PG	Washington Wizards	15.852

POSITION의 'PG' 중
SALARY가 15보다 높은 행 선택

■ 여러 컨디션 함수들

Predicate	Description
<code>are.equal_to(Z)</code>	Equal to Z
<code>are.above(x)</code>	Greater than x
<code>are.above_or_equal_to(x)</code>	Greater than or equal to x
<code>are.below(x)</code>	Less than x
<code>are.below_or_equal_to(x)</code>	Less than or equal to x
<code>are.between(x, y)</code>	Greater than or equal to x, and less than y
<code>are.strictly_between(x, y)</code>	Greater than x and less than y
<code>are.between_or_equal_to(x, y)</code>	Greater than or equal to x, and less than or equal to y
<code>are.containing(S)</code>	Contains the string S

```
nba.where('SALARY', are.between(10, 10.3))
```

PLAYER	POSITION	TEAM	SALARY
Luol Deng	SF	Miami Heat	10.1516
Gerald Wallace	SF	Philadelphia 76ers	10.1059
Danny Green	SG	San Antonio Spurs	10
DeMar DeRozan	SG	Toronto Raptors	10.05

SALARY의 10~10.3 사이의 행 선택

Predicate	Description
<code>are.not_equal_to(Z)</code>	Not equal to Z
<code>are.not_above(x)</code>	Not above x

```
nba.where('TEAM', are.containing('Warriors')).show()
```

PLAYER	POSITION	TEAM	SALARY
Klay Thompson	SG	Golden State Warriors	15.501
Draymond Green	PF	Golden State Warriors	14.2609
Andrew Bogut	C	Golden State Warriors	13.8
Andre Iguodala	SF	Golden State Warriors	11.7105

데이터 사이언스 모듈 (14) - 테이블

여러 컨디션 함수들

Predicate	Description
<code>are.equal_to(Z)</code>	Equal to <code>Z</code>
<code>are.above(x)</code>	Greater than <code>x</code>
<code>are.above_or_equal_to(x)</code>	Greater than or equal to <code>x</code>
<code>are.below(x)</code>	Less than <code>x</code>
<code>are.below_or_equal_to(x)</code>	Less than or equal to <code>x</code>
<code>are.between(x, y)</code>	Greater than or equal to <code>x</code> , and less than <code>y</code>
<code>are.strictly_between(x, y)</code>	Greater than <code>x</code> and less than <code>y</code>
<code>are.between_or_equal_to(x, y)</code>	Greater than or equal to <code>x</code> , and less than or equal to <code>y</code>
<code>are.containing(S)</code>	Contains the string <code>S</code>

Predicate	Description
<code>are.not_equal_to(Z)</code>	Not equal to <code>Z</code>
<code>are.not_above(x)</code>	Not above <code>x</code>

```
other_than_Cavs = nba.where('TEAM', are.not_equal_to('Cleveland Cavaliers'))
other_than_Cavs.where('SALARY', are.not_below(20))
```

PLAYER	POSITION	TEAM	SALARY
Joe Johnson	SF	Brooklyn Nets	24.8949
Derrick Rose	PG	Chicago Bulls	20.0931
Dwight Howard	C	Houston Rockets	22.3594

```
nba.where('TEAM', are.containing('Warriors')).show()
```

PLAYER	POSITION	TEAM	SALARY
Klay Thompson	SG	Golden State Warriors	15.501
Draymond Green	PF	nba.where('POSITION', are.containing('G'))	
Andrew Bogut	C		
Andre Iguodala	SF		
PLAYER	POSITION	TEAM	SALARY
Jeff Teague	PG	Atlanta Hawks	8
Kyle Korver	SG	Atlanta Hawks	5.74648
Dennis Schroder	PG	Atlanta Hawks	1.7634
Tim Hardaway Jr.	SG	Atlanta Hawks	1.30452
Jason Richardson	SG	Atlanta Hawks	0.947276
Lamar Patterson	SG	Atlanta Hawks	0.525093
Terran Petteway	SG	Atlanta Hawks	0.525093

```
other_than_Cavs.where('SALARY', are.above_or_equal_to(20))
```

PLAYER	POSITION	TEAM	SALARY
Joe Johnson	SF	Brooklyn Nets	24.8949
Derrick Rose	PG	Chicago Bulls	20.0931
Dwight Howard	C	Houston Rockets	22.3594

데이터 사이언스 모듈 (15) - 예제

■ (예제) 인구 동향

- <https://github.com/data-8/materials-sp26/blob/main/lec/lec06/nc-est2019-agesex-res.csv>

```
# As of August 2021, this census file is online here:
data = 'http://www2.census.gov/programs-surveys/popest/technical-documentation/file-layouts/2010-2019/nc-est2019-agesex-res.csv'

# A local copy can be accessed here in case census.gov moves the file:
# data = path_data + 'nc-est2019-agesex-res.csv'

full_census_table = Table.read_table(data)
full_census_table
```

SEX	AGE	CENSUS2010POP	ESTIMATESBASE2010	POPESTIMATE2010	POPESTIMATE2011	POPESTIMATE2012	POPESTIMATE2013	POPESTIMATE2014	POPESTIMATE2019
0	0	3944153	3944160	3951430	3963092	3926570	3931258	3931258	3
0	1	3978070	3978090	3957730	3966225	3977549	3942698	3942698	3
0	2	4096929	4096939	4090621	3970654	3978925	3991740	3991740	3
0	3	4119040	4119051	4111688	4101644	3981531	3991017	3991017	4
0	4	4063170	4063186	4077346	4121488	4111490	3992502	3992502	4

```
partial_census_table = full_census_table.select('SEX', 'AGE', 'POPESTIMATE2014', 'POPESTIMATE2019')
partial_census_table
```

SEX	AGE	POPESTIMATE2014	POPESTIMATE2019
0	0	3954787	3783052
0	1	3948891	3829599
0	2	3958711	3922044
0	3	4005928	3998665
0	4	4004032	4043323

데이터 사이언스 모듈 (16) - 예제

■ (예제) 인구 동향 (2)

```
us_pop = partial_census_table.relabelled('POPESTIMATE2014', '2014').relabelled('POPESTIMATE2019', '2019')
us_pop
```

SEX	AGE	2014	2019
0	0	3954787	3783052
0	1	3948891	3829599
0	2	3958711	3922044
0	3	4005928	3998665
0	4	4004032	4043323
0	5	4004576	4028281
0	6	4133372	4017227

```
us_pop_by_age = us_pop.where('SEX', are.equal_to(0)).drop('SEX')
us_pop_by_age
```

AGE	2014	2019
0	3954787	3783052
1	3948891	3829599
2	3958711	3922044
3	4005928	3998665
4	4004032	4043323
5	4004576	4028281

```
us_pop_by_age.where('AGE', are.between(97, 101))
```

AGE	2014	2019
97	82948	116969
98	59546	86150
99	41277	57124
100	70685	100322

데이터 사이언스 모듈 (17) - 예제

■ (예제) 인구 동향 (3)

```
change = us_pop_by_age.column('2019') - us_pop_by_age.column('2014')
change
```

```
array([-171735, -119292, -36667, -7263, 39291, 23705, -116145,
       -130347, -52155, -44194, -53618, 104804, 141200, 7126,
       -66894, 12631, -15505, -39515, 34483, 9650, -141227,
       -201943, -305343, -350651, -241835, 37812, 214055, 409073,
```

```
us_pop_change = us_pop_by_age.with_columns(
    'Change', change,
    'Percent Change', change/us_pop_by_age.column('2014')
)
us_pop_change.set_format('Percent Change', PercentFormatter)
```

AGE	2014	2019	Change	Percent Change
0	3954787	3783052	-171735	-4.34%
1	3948891	3829599	-119292	-3.02%
2	3958711	3922044	-36667	-0.93%
3	4005928	3998665	-7263	-0.18%

```
us_pop_change.where('AGE', are.equal_to(999))
```

AGE	2014	2019	Change	Percent Change
999	318301008	328239523	9938515	3.12%

```
us_pop_change.where(
    'AGE', are.below(999)
).sort('Change', descending=True)
```

AGE	2014	2019	Change	Percent Change
72	2191642	3191048	999406	45.60%
68	2567511	3345475	777964	30.30%
69	2530460	3252423	721963	28.53%
70	2461426	3136704	675278	27.43%

Q & A
